

同名不同人处理方案

2026年03月14日 v1.0

同名不同人处理方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、问题分析

1.1 典型场景

场景 1：同名不同人

- 张三（原告，1980 年出生，XXX 公司法人）
- 张三（证人，1995 年出生，自由职业者）

场景 2：同名 + 同电话（可能是同一人）

- 张三（电话：138****1234，原告）
 - 张三（电话：138****1234，证据签署人）
- 系统应提示"可能是同一人"

场景 3：同名 + 不同电话（需要人工判断）

- 张三（电话：138****1234，北京）
 - 张三（电话：139****5678，上海）
- 系统应标记为"待确认"

1.2 核心挑战

挑战	说明	解决思路
如何区分	同名但不同人	多字段组合识别

挑战	说明	解决思路
如何提示	避免误合并	置信度评分 + 人工确认
如何合并	确认是同一人后	合并功能 + 操作日志
如何拆分	误合并后	拆分功能 + 历史追溯

二、数据库设计补充

2.1 增加区分字段

```
-- 在 persons 表中增加区分字段
ALTER TABLE persons ADD COLUMN birth_date DATE;           -- 出生日期
ALTER TABLE persons ADD COLUMN birth_place TEXT;          -- 出生地
ALTER TABLE persons ADD COLUMN id_number_last4 TEXT;       -- 身份证后 4 位（脱敏）
ALTER TABLE persons ADD COLUMN avatar_url TEXT;            -- 头像照片
ALTER TABLE persons ADD COLUMN fingerprint TEXT;           -- 特征指纹（多字段哈希）

-- 人物识别信息表（扩展信息）
CREATE TABLE person_identifiers (
  id TEXT PRIMARY KEY,
  person_id TEXT NOT NULL REFERENCES persons(id),

  -- 识别类型
  identifier_type TEXT NOT NULL,           -- id number/phone/email/address/etc.
  identifier_value TEXT NOT NULL,          -- 识别值（加密）
  identifier_hash TEXT NOT NULL,           -- 哈希值（用于快速匹配）

  -- 验证状态
  is_verified BOOLEAN DEFAULT FALSE, -- 是否已验证
  verified_at DATETIME,
  verified_by TEXT,

  -- 来源
  source TEXT,           -- 来源: manual/evidence_extract/import
  confidence REAL DEFAULT 1.0, -- 置信度

  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```

-- 索引
INDEX idx_person ON person_identifiers(person_id),
INDEX idx_type_value ON person_identifiers(identifier_type, identifier_hash),
UNIQUE INDEX idx_unique (identifier_type, identifier_hash)
);

-- 人物合并记录表
CREATE TABLE person_merge_records (
    id TEXT PRIMARY KEY,
    kept_person_id TEXT NOT NULL,      -- 保留的人物 ID
    merged_person_id TEXT NOT NULL,    -- 被合并的人物 ID (原 ID)
    merge_reason TEXT NOT NULL,       -- 合并原因
    confidence REAL NOT NULL,         -- 合并置信度
    merged_by TEXT NOT NULL,
    merged_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    -- 可逆: 记录合并前的数据快照
    snapshot_before_merge JSON,

    INDEX idx_kept ON person_merge_records(kept_person_id),
    INDEX idx_merged ON person_merge_records(merged_person_id)
);

-- 人物相似度缓存表 (加速去重查询)
CREATE TABLE person_similarity_cache (
    person_a_id TEXT NOT NULL,
    person_b_id TEXT NOT NULL,
    similarity_score REAL NOT NULL,    -- 相似度 0-1
    matched_fields TEXT,              -- 匹配的字段

    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

    PRIMARY KEY (person_a_id, person_b_id),
    INDEX idx_score ON person_similarity_cache(similarity_score DESC)
);

```

2.2 特征指纹计算

```

// src/utils/person_fingerprint.rs
use sha2::{Sha256, Digest};

/// 计算人物特征指纹
/// 用于快速判断两个人是否可能相同
pub fn calculate_person_fingerprint(
    name: &str,

```

```

    birth_date: Option<&str>,
    phone: Option<&str>,
    org: Option<&str>,
) -> String {
    let mut hasher = Sha256::new();

    // 姓名 (标准化后)
    hasher.update(name.trim().to_lowercase().as_bytes());

    // 出生日期 (如有)
    if let Some(birth) = birth_date {
        hasher.update(birth.as_bytes());
    }

    // 电话 (标准化后)
    if let Some(p) = phone {
        hasher.update(normalize_phone(p).as_bytes());
    }

    // 单位 (标准化后)
    if let Some(o) = org {
        hasher.update(o.trim().to_lowercase().as_bytes());
    }

    let result = hasher.finalize();
    format!("{:x}", result)
}

/// 标准化电话号码
fn normalize_phone(phone: &str) -> String {
    // 移除空格、横线、括号
    phone.chars()
        .filter(|c| c.is_ascii_digit() || c == &'+' || c == &'#')
        .collect()
}

```

三、同名识别策略

3.1 多层次识别

```
// src/services/person_identification.rs

/// 同名识别结果
#[derive(Debug, Clone)]
pub struct SameNameIdentification {
    pub person_id: String,
    pub name: String,
    pub is_same_person: bool,    // 是否同一人
    pub confidence: f64,        // 置信度 0-1
    pub matched_fields: Vec<String>, // 匹配的字段
    pub different_fields: Vec<String>, // 不同的字段
    pub action_required: ActionRequired, // 需要采取的行动
}

#[derive(Debug, Clone, PartialEq)]
pub enum ActionRequired {
    AutoMerge,    // 自动合并 (置信度>0.95)
    SuggestMerge, // 建议合并 (置信度 0.7-0.95)
    ManualReview, // 人工审核 (置信度 0.3-0.7)
    ConfirmSeparate, // 确认是不同人 (置信度<0.3)
}

/// 识别服务
pub struct PersonIdentificationService {
    pool: SqlitePool,
}

impl PersonIdentificationService {
    /// 检查新人物是否与现有人物重复
    pub async fn check_duplicate(
        &self,
        input: &CreatePersonInput,
    ) -> anyhow::Result<Vec<SameNameIdentification>> {
        let mut results = Vec::new();

        // 1. 精确匹配 (身份证/电话哈希)
        if let Some(exact_match) = self.find_exact_match(input).await? {
            results.push(SameNameIdentification {
                person_id: exact_match.id,
                name: exact_match.name,
                is_same_person: true,
            });
        }
    }
}
```

```

        confidence: 1.0,
        matched_fields: vec!["id_number".to_string()],
        different_fields: vec![],
        action_required: ActionRequired::AutoMerge,
    });
    return Ok(results); // 精确匹配, 直接返回
}

// 2. 查找同名人物
let same_name_persons = self.find_same_name_persons(&input.name).await?;

for person in same_name_persons {
    let (confidence, matched, different) =
        self.calculate_similarity(&person, input);

    let action = if confidence > 0.95 {
        ActionRequired::AutoMerge
    } else if confidence > 0.7 {
        ActionRequired::SuggestMerge
    } else if confidence > 0.3 {
        ActionRequired::ManualReview
    } else {
        ActionRequired::ConfirmSeparate
    };

    results.push(SameNameIdentification {
        person_id: person.id,
        name: person.name.clone(),
        is_same_person: confidence > 0.5,
        confidence,
        matched_fields: matched,
        different_fields: different,
        action_required: action,
    });
}

// 按置信度排序
results.sort_by(|a, b| b.confidence.partial_cmp(&a.confidence).unwrap());

Ok(results)
}

/// 计算相似度
fn calculate_similarity(
    &self,
    existing: &Person,
    input: &CreatePersonInput,

```

```

) -> (f64, Vec<String>, Vec<String>) {
    let mut score = 0.0;
    let mut total_weight = 0.0;
    let mut matched = Vec::new();
    let mut different = Vec::new();

    // 姓名相同 (基础分, 权重 0)
    // 已经过滤同名, 这里是计算其他字段

    // 出生日期 (权重 30%)
    total_weight += 0.3;
    if let (Some(e_birth), Some(i_birth)) = (&existing.birth_date, &input.birth_date) {
        if e_birth == i_birth {
            score += 0.3;
            matched.push("birth_date".to_string());
        } else {
            different.push("birth_date".to_string());
        }
    }

    // 电话 (权重 25%)
    total_weight += 0.25;
    if let (Some(e_phone), Some(i_phone)) = (&existing.phone, &input.phone) {
        if self.normalize_phone(e_phone) == self.normalize_phone(i_phone) {
            score += 0.25;
            matched.push("phone".to_string());
        } else {
            different.push("phone".to_string());
        }
    }

    // 单位 (权重 20%)
    total_weight += 0.2;
    if let (Some(e_org), Some(i_org)) = (&existing.organization, &input.organization) {
        let similarity = self.string_similarity(&e_org.to_lowercase(), &i_org.to_lowercase());
        if similarity > 0.8 {
            score += 0.2 * similarity;
            matched.push("organization".to_string());
        } else {
            different.push("organization".to_string());
        }
    }

    // 地址 (权重 15%)
    total_weight += 0.15;
    if let (Some(e_addr), Some(i_addr)) = (&existing.address, &input.address) {
        let similarity = self.string_similarity(&e_addr.to_lowercase(), &i_addr.to_lowerca

```

```

        if similarity > 0.6 {
            score += 0.15 * similarity;
            matched.push("address".to_string());
        } else {
            different.push("address".to_string());
        }
    }

    // 身份证后 4 位 (权重 10%)
    total_weight += 0.1;
    if let (Some(e_last4), Some(i_last4)) =
        (&existing.id_number_last4, &input.id_number_last4) {
        if e_last4 == i_last4 {
            score += 0.1;
            matched.push("id_last4".to_string());
        } else {
            different.push("id_last4".to_string());
        }
    }
}

let final_score = if total_weight > 0.0 {
    score / total_weight
} else {
    0.5 // 无其他信息, 默认中等置信度
};

(final_score, matched, different)
}

/// 字符串相似度 (Levenshtein 距离)
fn string_similarity(&self, a: &str, b: &str) -> f64 {
    let len_a = a.len();
    let len_b = b.len();

    if len_a == 0 && len_b == 0 {
        return 1.0;
    }
    if len_a == 0 || len_b == 0 {
        return 0.0;
    }

    let distance = self.levenshtein_distance(a, b);
    1.0 - (distance as f64 / len_a.max(len_b) as f64)
}
}

```


3.2 识别规则总结

匹配情况	置信度	处理方式
身份证号相同	100%	自动合并
电话相同 + 姓名相同	95%	建议合并，用户确认
姓名 + 生日 + 单位相同	90%	建议合并，用户确认
姓名 + 电话不同	80%	提示人工审核
姓名 + 单位不同	60%	提示人工审核
仅姓名相同	50%	标记"待确认"
姓名 + 生日不同	30%	确认为不同人

四、前端交互设计

4.1 创建人物时的查重提示

```
// frontend/src/components/person/CreatePersonModal.rs

#[component]
pub fn CreatePersonModal(
    case_id: String,
    on_close: EventHandler<()>,
    on_submit: EventHandler<CreatePersonInput>,
) -> Element {
    let mut form = use_signal(|| CreatePersonInput::default());
    let duplicate_check = use_resource(move || {
        let name = form().name;
        if name.len() > 1 {
            async move { check_duplicate_persons(&name).await.ok() }
        } else {
            async move { None }.await
        }
    });
}
```

```

    }
  });

  rsx! {
    div { class: "modal-overlay",
      div { class: "modal create-person-modal",
        h2 { "添加人物" }

        // 基本信息表单
        input {
          placeholder: "姓名 *",
          value: form().name,
          oninput: move |e| form.write().name = e.value(),
        }

        input {
          placeholder: "电话",
          value: form().phone(),
          oninput: move |e| form.write().phone = Some(e.value()),
        }

        input {
          placeholder: "身份证号",
          value: form().id_number(),
          oninput: move |e| form.write().id_number = Some(e.value()),
        }

        input {
          placeholder: "工作单位",
          value: form().organization(),
          oninput: move |e| form.write().organization = Some(e.value()),
        }

        // 重复检测结果
        if let Some(duplicates) = &*duplicate_check.value() {
          if !duplicates.is_empty() {
            div { class: "duplicate-warning",
              h4 { "⚠️ 发现相似人物" }

              for dup in duplicates {
                div {
                  class: "duplicate-item {dup.action_required}",
                  onclick: move |_| view_person_detail(&dup.person_id),

                  div { class: "person-info",
                    strong { "{dup.name}" }
                    span { class: "confidence",

```



```

// 合并箭头
div { class: "merge-arrow", "→ 合并 →" }

// 人物 B
div { class: "person-column merge",
  h3 { "将被合并" }
  match &*person_b.value() {
    Some(Ok(p)) => rsx! {
      PersonSummary { person: p.clone() }
    },
    _ => rsx! { div { "加载中..." } }
  }
}

// 合并后效果说明
div { class: "merge-info",
  h4 { "合并后将：" }
  ul {
    li { "保留人物 A 的基本信息" }
    li { "人物 B 的所有关联（案件/证据/节点）将转移至人物 A" }
    li { "人物 B 标记为'已合并'，不再单独显示" }
    li { "操作记录将永久保存，支持追溯" }
  }
}

// 合并原因
textarea {
  placeholder: "合并原因（选填）",
  rows: 3,
}

// 确认
div { class: "warning",
  "⚠️ 此操作不可逆，请确认两人确为同一人"
}

div { class: "modal-actions",
  button { class: "cancel", onclick: move |_| on_close.call(()), "取消" }
  button {
    class: "submit danger",
    onclick: move |_| execute_merge(&person_a_id, &person_b_id,
    "确认合并"
  )
}
}
}
}

```

```
}  
}
```

4.3 人物拆分功能

```
// frontend/src/components/person/SplitPersonModal.rs  
  
#[component]  
pub fn SplitPersonModal(  
    person_id: String,  
    on_close: EventHandler<()>,  
) -> Element {  
    let person = use_resource(move || fetch_person_with_relations(&person_id));  
    let mut selected_links = use_signal(|| Vec::new());  
  
    rsx! {  
        div { class: "modal-overlay",  
            div { class: "modal split-person-modal",  
                h2 { "拆分人物" }  
  
                p { class: "description",  
                    "选择需要拆分出去的关联，系统将创建新的人物记录并转移这些关联。"  
                }  
  
                // 当前人物信息  
                match &*person.value() {  
                    Some(Ok(p)) => rsx! {  
                        div { class: "current-person",  
                            h3 { "当前人物: {p.person.name}" }  
                            div { class: "stats",  
                                span { "📁 {p.case_links.len()} 案件" }  
                                span { "📄 {p.evidence_links.len()} 证据" }  
                                span { "🔗 {p.person_relations.len()} 人物关系" }  
                            }  
                        }  
                    }  
                }  
  
                // 选择要拆分的关联  
                div { class: "split-selection",  
                    h4 { "选择要拆分出去的关联" }  
  
                    // 案件关联  
                    div { class: "section",  
                        h5 { "案件关联" }  
                        for link in p.case_links.iter() {  
                            label { class: "checkbox-item",
```

```

        input {
            r#type: "checkbox",
            onchange: move |e| {
                if e.checked {
                    selected_links.write().push(link.id.clone())
                }
            },
        },
    },
    span { "{link.role_type} - 案件名称" }
}
}
}

// 证据关联
div { class: "section",
    h5 { "证据关联" }
    for link in p.evidence_links.iter() {
        label { class: "checkbox-item",
            input {
                r#type: "checkbox",
                onchange: move |e| {
                    if e.checked {
                        selected_links.write().push(link.id.clone())
                    }
                },
            },
            span { "{link.evidence_name}" }
        }
    }
}

// 新人物信息
div { class: "new-person-form",
    h4 { "新人物信息" }
    input { placeholder: "姓名", defaultValue: p.person.name.clone() }
    input { placeholder: "电话" }
    input { placeholder: "身份证号" }
}

},
_ => rsx! { div { "加载中..." } }
}

div { class: "modal-actions",
    button { class: "cancel", onclick: move |_| on_close.call(()), "取消" }
    button {
        class: "submit warning",
    }
}

```



```
}  
}
```

5.2 拆分人物

```
POST /api/v1/persons/:id/split
```

请求体:

```
{  
  "new_person": {  
    "name": "张三",  
    "phone": "139****5678",  
    "birth_date": "1995-01-01"  
  },  
  "links_to_transfer": [  
    {"type": "case_link", "id": "link_001"},  
    {"type": "evidence_link", "id": "link_002"}  
  ]  
}
```

响应:

```
{  
  "code": 200,  
  "data": {  
    "original_person_id": "person_001",  
    "new_person_id": "person_003",  
    "transferred_links": 2  
  }  
}
```

5.3 获取合并历史

```
GET /api/v1/persons/:id/merge-history
```

响应:

```

{
  "code": 200,
  "data": {
    "merges": [
      {
        "merged_person_id": "person_002",
        "merged_at": "2026-03-14T10:30:00Z",
        "merged_by": "user_001",
        "reason": "同一人，电话和生日相同"
      },
      {
        "merged_person_id": "person_004",
        "merged_at": "2026-03-10T14:00:00Z",
        "merged_by": "user_001",
        "reason": "同一人，身份证相同"
      }
    ]
  }
}

```

六、操作日志记录

```

// src/services/audit_person.rs

/// 记录人物相关操作
pub async fn log_person_action(
    ctx: &AppContext,
    action: PersonAction,
    person_id: &str,
    details: serde_json::Value,
) {
    log_operation!(
        ctx,
        action: action.to_action_type(),
        module: ModuleType::Person,
        target_type: "person",
        target_id: person_id.to_string(),
        target_title: details["name"].as_str().map(|s| s.to_string()),
        old: details.get("old_value").cloned(),
        new: details.get("new_value").cloned()
    );
}

```

```
}

/// 人物操作类型
#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum PersonAction {
    Create,
    Update,
    Delete,
    Merge,           // 合并他人
    BeMerged,        // 被合并
    Split,           // 拆分
    LinkCase,
    UnlinkCase,
    LinkEvidence,
    LinkPerson,
}
```

七、总结

7.1 核心策略

策略	说明
多层次识别	身份证 > 电话 > 生日 > 单位 > 地址
置信度评分	0-1 分，>0.95 自动合并，0.7-0.95 建议合并
人工确认	中等置信度时提示用户判断
可逆操作	合并记录永久保存，支持拆分追溯
前端提示	创建/编辑时实时查重

7.2 处理流程

创建/编辑人物
↓

查重检测（同名 + 多字段匹配）



置信度 > 0.95 → 自动合并，记录日志	
置信度 0.7-0.95 → 建议合并，用户确认	
置信度 0.3-0.7 → 人工审核，判断是否同人	
置信度 < 0.3 → 标记为不同人	



完成创建/更新

7.3 数据量估算

场景	频率	存储
查重检测	每次创建/编辑	缓存 7 天
合并操作	<1 次/案件	永久保存记录
拆分操作	罕见	永久保存记录
相似度缓存	实时计算	保留 30 天

文档版本：v1.0

最后更新：2026 年 3 月 14 日

开发负责人：王舟

邮箱：main@mails.wedevs.org

电话：15378391447

开发负责人：王舟 | 电话：15378391447 | 邮箱：main@mails.wedevs.org